

Onde o nosso kit está, comparado ao mercado

Segunda rodada — treze alternativas, doze critérios congelados antes da pesquisa, cétricos encarregados de derrubar as notas altas, e a evidência independente que aponta contra nós

Documento preparado para apresentação em equipe · 6 de setembro de 2026, revisado em 10 de setembro com quatro correções sobre o próprio kit · critérios congelados sob o hash sha256 9d46b98b...

Como ler este documento

Este é o segundo benchmarking do kit de processo contra as alternativas de mercado. Ele existe porque o kit tinha nota alta em disciplina e nota baixa em **prova de que funciona** — e um benchmarking mal-feito piora exatamente esse problema, porque fabrica confiança onde não há medida. As regras abaixo são o que separa este documento de um folheto.

Todo número tem um rótulo de evidência

- **[reproduzido]** — eu rodei o comando e coleí a saída. **Só o nosso kit recebe este rótulo.**
- **[derivado]** — eu li a documentação oficial, o repositório ou uma análise independente.
- **[suposto]** — memória, não confirmada. Aparece marcado, e vale menos.

“Não encontrei” nunca virou “não existe”. Onde a pesquisa não achou o mecanismo, a ferramenta levou a nota do que foi possível confirmar e a incerteza ficou escrita.

A régua, a mesma para todo mundo

- **0** não existe · **1** só menção · **2** existe, manual e frágil
- **3** existe e funciona, mas depende de disciplina humana
- **4** existe e é **cobrado por máquina** (bloqueia commit, falha o CI, nega a chamada)
- **5** cobrado por máquina e medido em uso real, com número publicado

Nota 4 exigiu mecanismo automático verificável. Documentação que apenas *diz* que faz parou em 3.

Nota 5 exigiu medição publicada de desfecho — praticamente ninguém tem, **nós inclusive**.

A assimetria de evidência

Isto não é vantagem nossa. É o contrário.

Eu **medi** o nosso kit: rodei o portão, rodei os 141 testes, li o git de um projeto real de 187 commits. Eu **não consegui medir nenhum concorrente** — não existe projeto real meu feito com nenhum deles, então todos foram julgados pela documentação.

Defeito só aparece em uso. Logo, o lado medido aparece **com os defeitos à mostra** e o lado lido aparece **pela promessa**. Um 3 que dei a um concorrente pode ser um 4 que a documentação não mostrou; um 4 que dei a ele pode ser um 2 que só o uso revelaria. As duas direções estão abertas, e nenhuma delas é simétrica com o nosso caso.

Conflito de interesse, declarado

- Quem escreveu os critérios é o mesmo agente que ajudou a construir o kit avaliado.
- A pesquisa rodou **dentro do Claude Code**, que é uma das ferramentas avaliadas. O avaliador é cliente de um dos avaliados.
- O dono do kit é quem pediu o benchmark e é o beneficiário do resultado.
- Autor, avaliador e beneficiário sendo a mesma pessoa, com **um** projeto e **sem grupo de controle**, é o degrau mais baixo da escala de evidência. Um estudo independente com resultado ruim valeria mais que esta auto-medição com resultado bom.

O erro de categoria que este documento evita

Claude Code, Codex CLI, Gemini CLI, Aider e Devin são **ferramentas de agente**. O nosso kit **roda em cima** de uma delas. Compará-los como pares é comparar o andar com o alicerce. Eles aparecem no placar em **cinza**, como linha de base, e somar as notas deles contra as nossas é leitura errada — a linha de base existe para responder “o que já vem de graça na caixa?”, não “quem ganha?”.

O que foi congelado antes de pesquisar

Os doze critérios foram escritos e hashados **antes** de a primeira busca sair. Benchmark que escolhe o critério depois de ver o resultado não vale nada.

sha256 do arquivo de critérios: **9d46b98b78b67cfb5d752eaba42ca9f4ffb5f530bc14ac18e9027ec9cae27181**
congelado em 2026-09-05 23:17:14 (local) · 2026-09-06 02:17:13 UTC

O teste de falseamento

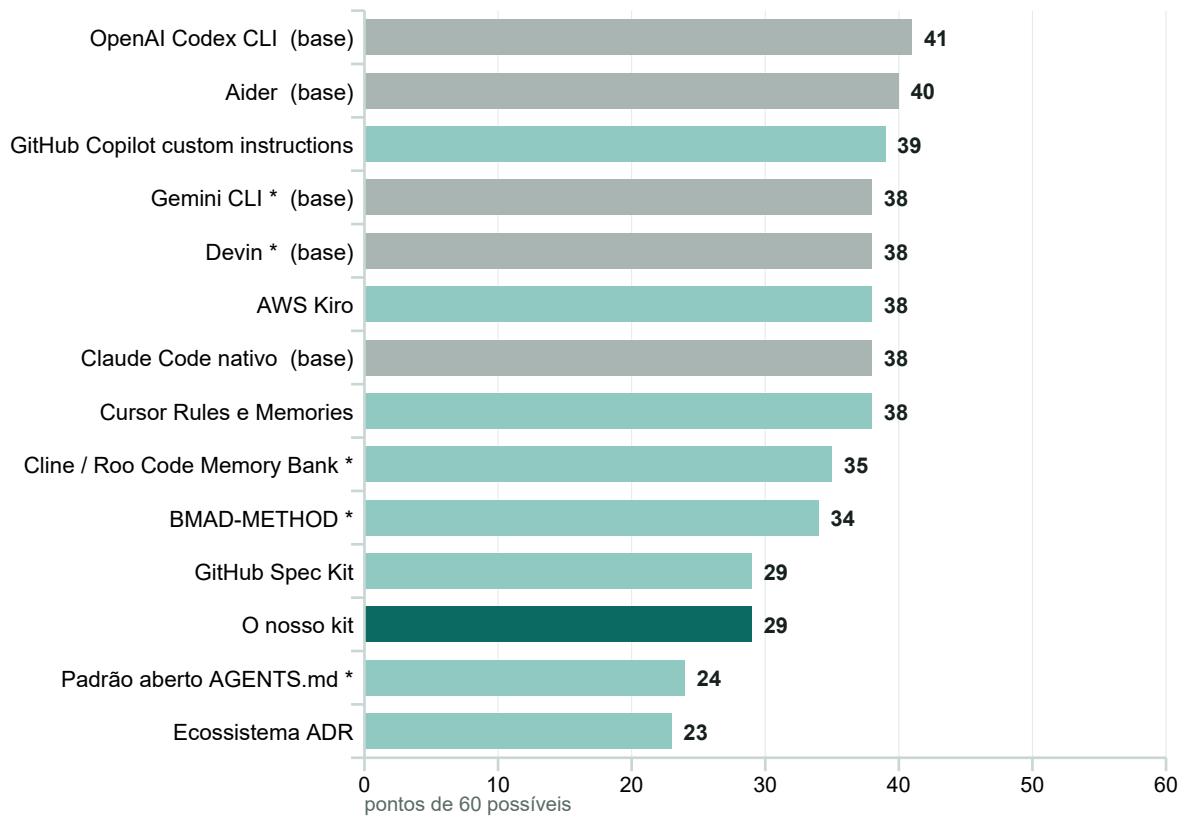
A rodada anterior tinha um defeito grave: **não existia critério em que o kit pudesse perder feio**. Quatro dos dez descreviam quase literalmente o que o kit faz, e um benchmark assim é auto-retrato. Esta rodada mudou três coisas:

- **Fundiu** dois critérios que davam ao kit dois pontos pelo mesmo mecanismo (“economia de contexto” e “escala” viraram um só).
- **Acrescentou três critérios novos** — segurança e isolamento (C10), distribuição em equipe (C11) e desfazer/recuperar (C12) — escolhidos exatamente porque se espera que o kit perca neles.
- **Marcou C2 e C8** como os dois critérios que ainda são parecidos com auto-retrato. Eles ficaram porque são legítimos, mas **leia essas duas colunas com desconto**.

Ao todo, **sete dos doze** critérios foram escritos esperando que o kit perdesse: C5, C6, C7, C9, C10, C11 e C12. Se o kit tivesse ganhado em todos, o benchmark estaria errado, e este documento diria isso.

O placar

Treze alternativas, doze critérios, uma régua só. O nosso kit soma **29 de 60** — e este documento **não vai dizer em que lugar ele ficou**, porque o número que faria essa frase valer não existe: **5 das 13 alternativas não passaram pelo cético**, e as notas altas delas continuam não verificadas.



Soma dos 12 critérios, já com as auditorias aplicadas. Cinza = linha de base (ferramenta de agente; o kit roda em cima dela). * = notas 4+ não auditadas.

Três motivos para não ler o total.

- 1. A auditoria foi desigual.** As 8 ferramentas auditadas perderam **1.5 ponto em média**; as 5 não auditadas perderam zero — não por estarem certas, mas por ninguém ter olhado. O nosso kit perdeu **3 pontos**, o dobro da média.
- 2. Um terço da escala mede a mesma coisa.** C9, C10, C11 e C12 juntos perguntam, no fundo, “existe uma empresa por trás?”. Contra isso um projeto de uma pessoa perde quatro vezes pelo mesmo motivo.
- 3. Doze notas de peso igual** fazem portabilidade valer o mesmo que portão. Ninguém decide assim, e quem reordenar os pesos inverte o ranking em dois minutos.

O placar, ferramenta por critério

Ferramenta	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	Total
O nosso kit	4	3	4	3	2	1	2	3	1	2	2	2	29
OpenAI Codex CLI (base)	3	2	4	4	3	4	4	3	4	4	4	2	41
Aider (base)	4	2	3	3	4	4	4	4	3	2	3	4	40
GitHub Copilot custom instructions	3	2	2	4	2	4	3	3	4	4	4	4	39
Cursor Rules e Memories	3	2	4	4	1	3	2	3	4	4	4	4	38
Claude Code nativo (base)	3	2	4	4	1	3	2	3	4	4	4	4	38
AWS Kiro	4	2	4	4	1	2	2	3	4	4	4	4	38
Devin * (base)	3	2	3	4	4	2	1	3	4	4	4	4	38
Gemini CLI * (base)	4	2	4	4	1	3	2	3	3	4	4	4	38
Cline / Roo Code Memory Bank *	2	3	1	4	1	3	4	2	4	4	3	4	35
BMAD-METHOD *	3	3	3	4	1	1	4	2	4	2	4	3	34
GitHub Spec Kit	2	3	2	3	3	2	4	2	3	1	3	1	29
Padrão aberto AGENTS.md *	2	1	1	0	3	4	4	2	4	0	3	0	24
Ecosistema ADR	1	3	1	1	1	4	4	1	3	0	3	1	23

Verde = 4 ou 5 (cobrado por máquina) · laranja = 3 (funciona, exige disciplina) · vermelho = 0 a 2. * = ferramenta cujas notas 4+ **não** foram auditadas — leia-as como teto, não como valor. Toda nota das 13 é **[derivado]**; toda nota do kit é **[reproduzido]**.

A leitura que o total esconde

O revisor adversarial fez a conta que faltava: separar os oito critérios que medem **capacidade de processo** (C1 a C8) dos quatro que medem, na prática, **se existe uma empresa por trás** (C9 manutenção, C10 segurança, C11 equipe, C12 desfazer). O resultado reposiciona o documento inteiro.

Ferramenta	C1–C8 processo	C9–C12 fornecedor	Total
Aider	28	12	40
OpenAI Codex CLI	27	14	41
GitHub Copilot custom instructions	23	16	39
Gemini CLI	23	15	38
Cursor Rules e Memories	22	16	38
Claude Code nativo	22	16	38
AWS Kiro	22	16	38
Devin	22	16	38
O NOSSO KIT	22	7	29
BMAD-METHOD	21	13	34
GitHub Spec Kit	21	8	29
Cline / Roo Code Memory Bank	20	15	35
Padrão aberto AGENTS.md	17	7	24
Ecossistema ADR	16	7	23

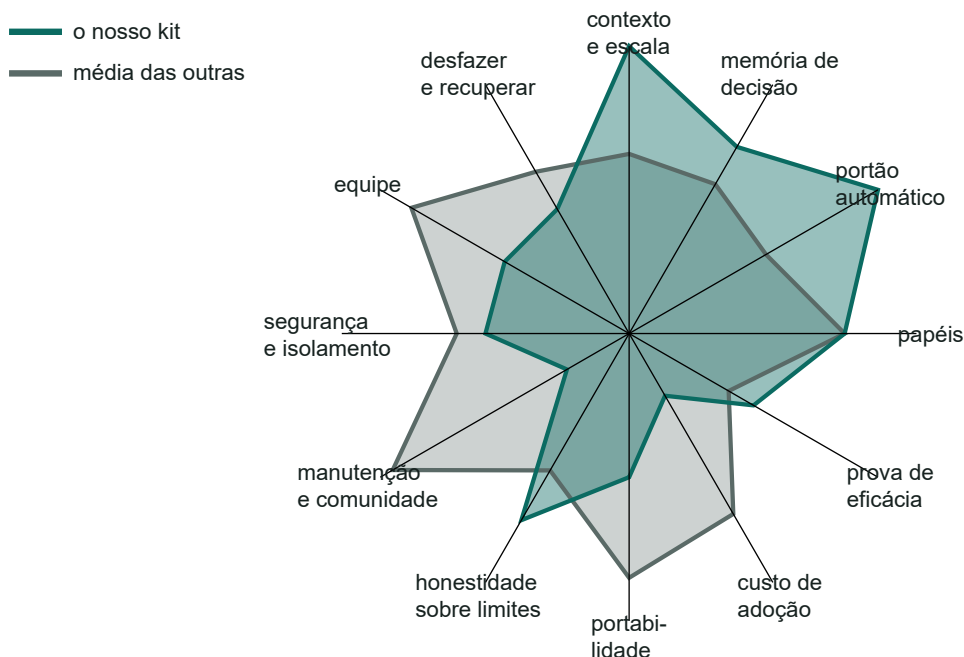
O kit compete no mecanismo e perde no que faz uma coisa sobreviver a uma equipe.

Em capacidade de processo (C1–C8) o kit soma **22 de 40** e fica em **5º**, empatado com 4 outras — acima de BMAD, Spec Kit, Memory Bank, AGENTS.md e do próprio ecossistema ADR. Em C9–C12 soma **7 de 20**, contra 15 ou 16 de quase todo produto comercial.

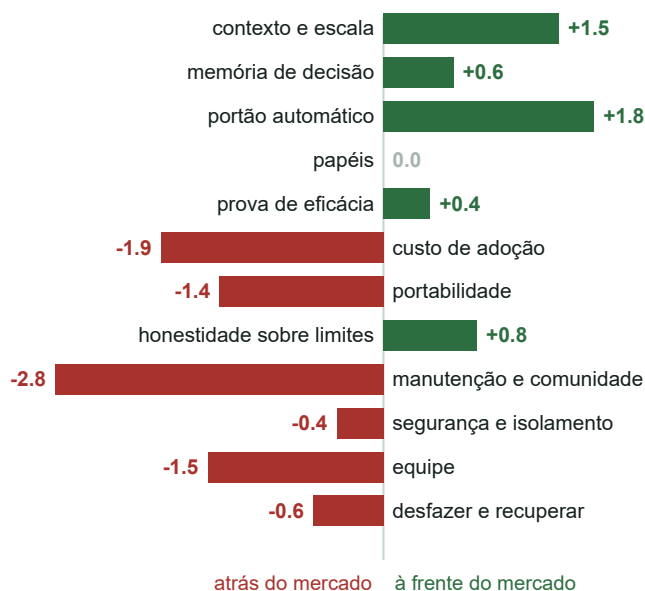
Isso corta nos dois sentidos, e o documento precisa dizer os dois. **A favor:** a distância do kit para o topo do placar não é falta de mecanismo — é ausência de fornecedor. **Contra:** ausência de fornecedor é exatamente o que decide se a coisa sobrevive a férias, a uma demissão e à sexta pessoa entrando no time.

O formato, e não o tamanho

O kit não é uma ferramenta mediana: ele é **pontudo** — alto em quatro coisas, muito baixo em cinco.



O nosso kit contra a média das alternativas que competem no mesmo nível.



Nós menos a média das concorrentes, critério a critério. Verde à direita = à frente.

Os dois gráficos comparam com a MÉDIA, e média favorece todo mundo.

Contra a **melhor de cada critério** — que é com quem a equipe compararia de verdade na hora de escolher — o kit fica atrás em **8 dos 12** (C4, C5, C6, C7, C9, C10, C11, C12), empata em 4 (C1, C2, C3, C8) e não fica à frente em nenhum. Sempre que estes gráficos forem para um slide, esta ressalva vai junto.

O que a auditoria derrubou

Toda nota 4 ou 5 deveria passar por um cético dedicado, com busca na web, encarregado de procurar o mecanismo e rebaixar a nota se ele não existisse ou fosse só recomendação. **8 das 13** passaram — a cota de sessão acabou antes das outras 5. As que passaram perderam pontos; as que não passaram aparecem no placar com asterisco.

A assimetria da auditoria, em número

Auditadas: **8**, variação média **-1.5** ponto. Não auditadas: **5** (BMAD-METHOD, Cline / Roo Code Memory Bank, Padrão aberto AGENTS.md, Devin, Gemini CLI), variação **0** — porque ninguém olhou, não porque estavam certas.

O nosso kit foi auditado e perdeu **3 pontos** (32 → 29), o dobro da média das concorrentes. Isso **não** significa que o kit é o pior: significa que ele foi olhado de perto, e o olhar de perto custa pontos em qualquer um. É a mesma assimetria da página 2, agora medida.

As mudanças, uma a uma

Ferramenta	Crit.	Nota	Por quê
GitHub Spec Kit	C9	4 → 3	REBAIXO POR AUSÊNCIA DE MECANISMO QUE COBRE O CRITÉRIO, NÃO POR FALTA DE VERIFICAÇÃO — verifiquei a fundo e os fatos do pesquisador conferem (API: MIT, criado 21/ago/2025, v1.0.0 em 21/ago/2026, v1.0.4 em 02/se...
Cursor Rules e Memories	C6	4 → 3	Não existe orçamento cobrado por máquina: a única fronteira que a doc de rules enuncia é "Keep rules under 500 lines", explicitamente uma best practice, sem limite técnico que valide, avise ou bloqueie. O peso ...
Claude Code nativo	C6	4 → 3	REBAIXADA APOS VERIFICACAO, nao por falta de busca: o que e automatico e so o primeiro arquivo (`/init` gera o CLAUDE.md analisando o repo), enquanto o custo recorrente — o proprio "peso por sessao" da justific...
Claude Code nativo	C8	4 → 3	REBAIXADA APOS VERIFICACAO, nao por falta de busca — conferi cada citacao e todas existem ao pe da letra ("Claude treats them as context, not enforced configuration" e "there's no guarantee of strict compliance...
Aider	C7	5 → 4	Todos os fatos conferem e são reais no código, não promessa: Apache-2.0, roda em terminal sem IDE, camada de provedores que aceita 15+ serviços mais endpoints OpenAI-compatíveis e modelos locais via Ollama, e C...
GitHub Copilot custom instructions	C8	4 → 3	As negativas existem e eu as confirmei verbatim na doc do VS Code ("Custom instructions are not taken into account for inline suggestions as you type in the editor"; "no specific order is guaranteed"; "Nested A...
GitHub Copilot custom instructions	C9	5 → 4	Todas as datas do pesquisador conferem (changelog github.blog: instruções de organização GA em 2026-04-02, firewall de organização em 2026-04-03, Agent Plugins 1.0 em 2026-08-12; repo oficial github/awesome-cop...
OpenAI Codex CLI	C1	4 → 3	O teto existe e é aplicado por código (project_doc_max_bytes, default 32 KiB), mas o próprio comentário do fonte diz que arquivos maiores são "silently truncated": nada bloqueia, nada falha, nada avisa — a issu...
OpenAI Codex CLI	C8	4 → 3	As autodeclarações existem e são exatas — a doc diz do proxy "It does not filter web search, app or connector tool calls, MCP server connections, browser or Computer Use activity", do DNS "it does not eliminate...

Ferramenta	Crit.	Nota	Por quê
OpenAI Codex CLI	C9	5 → 4	Confirmei os números (Apache-2.0, cadência alta, 122 mil estrelas hoje, e o anúncio da OpenAI de 4 milhões de desenvolvedores semanais em abril/2026), mas TODOS são adoção — exatamente o que a régua exclui como...
Ecossistema ADR	C6	5 → 4	Achei automação real e verificável (adr-tools: `adr new -s 9` numera sozinho e vira o status do ADR superado; MADR: `npm install madr && cp node_modules/madr/template/*`), mas nada trava commit nem falha CI no ...
Ecossistema ADR	C7	5 → 4	Confirmei o artefato portátil (markdown puro versionado, legível sem ferramenta nenhuma; Log4brains consome ADR criado pelo adr-tools), mas nada NO ecossistema cobra isso por máquina: MADR entrega só um `.markd...

Um defeito do instrumento que não dá para maquiar

Cursor Rules e Claude Code nativo terminaram com os doze dígitos idênticos.

3-2-4-4-1-3-2-3-4-4-4-4, dígito por dígito. Como resultado de duas medições independentes isso é praticamente impossível, e o revisor adversarial apontou com razão: a régua está **uniformizando em vez de discriminar**. Cinco das treze ferramentas caem em 38 ou 38-1.

Eu poderia inventar uma diferença para desfazer o empate. Não vou: a diferença que eu escrevesse agora não teria sido pesquisada, e um número inventado é pior que um empate constrangedor. Fica registrado como **limitação da régua**, e é o primeiro item a consertar na rodada 3.

O nosso kit, medido

Estas são as únicas notas **[reproduzido]** do documento: cada uma sai de um comando que eu rodei, com a saída colada. Três delas foram rebaixadas por um auditor que leu o código-fonte do kit e achou o que eu não tinha achado.

Errata de 10 de setembro: quatro afirmações sobre o nosso kit estavam erradas.

Preparando as melhorias que este documento motivou, reli o changelog do kit e medi o projeto real de novo — e a primeira versão deste documento não sobreviveu inteira.

1. Eu disse que o kit **não tinha caminho de atualização**. Tem, desde a v9: `new_project.py --upgrade`, com marca de versão — e, desde a v10, manifesto de customização. C11 sobe de 1 para 2.
2. Eu chamei de **divergência** os dois tetos elevados do projeto real. São configuração declarada, prevista pelo kit. A deriva de verdade é de **versão**: o projeto está na v13.12, anterior ao conserto do QA-15 da v13.16.
3. Eu pus **13,7 minutos** no custo de adoção. É a suíte do mantenedor, que não roda em projeto nenhum; o portão custa de **2 a 5 s** por commit.
4. Eu creditei ao kit **três travas** em uso. No projeto real roda **uma**.

As três primeiras favoreciam os concorrentes; só a última favorecia o kit. O total vai de 28 para 29. Nenhuma foi pega pelos revisores: eles não receberam o changelog do kit, e eu não o tinha lido.

Os comandos, e o que eles devolveram

- `python scripts/check.py` → código 0, com 6 avisos.
- `python scripts/test_check.py` → **141 testes, todos verdes, em 821 segundos**.
- `task.py` evidencia no projeto real (187 commits, 11 dias) → 112 decisões, **11 rejeitadas** (9,8%), 44 achados de QA (8 críticos, 42 fechados), 15 questões ao dono, 13 das 24 skills usadas, **86,6% dos commits citam um ID**, orçamento em 3.653/4.000 (91,3%).
- `task.py` evidencia no repositório do próprio kit → 43 commits, **18,6% citam um ID**, e **25 dos últimos 30 commits fora do padrão que o kit exige**.

As doze notas

	Critério	Nota	Por quê — e o que o auditor disse
C1	contexto e escala	4	Orçamento de 4.000 caracteres cobrado pelo pre-commit; o projeto real bate 91,3% do teto. Confirmado pelo auditor , que leu o <code>check.py</code> e achou a trava que exige uma decisão registrada para elevar o teto. Não é 5 porque cobra o tamanho do arquivo, não o contexto que a sessão gastou — limite que o próprio relatório do kit declara.
C2	memória de decisão	3	Rebaixado de 4 para 3 . A máquina cobra a integridade do <i>ponteiro</i> (ID citado que não existe, ID duplicado), não o <i>ato</i> de registrar: o hook de mensagem não exige ID nenhum, e “ID prometido e nunca registrado” é aviso, não reprovação. 112 decisões e 11 rejeições no projeto real são disciplina do autor, como no ADR.

	Critério	Nota	Por quê — e o que o auditor disse
C3	portão automático	4	Confirmado. O pre-commit devolve código de bloqueio (18 falhas possíveis, 141 testes verdes) — e é a única trava ligada no projeto real: o hook de mensagem, o de escopo e o do pulo existem no kit e não estão instalados lá. Os 86,6% de commits com ID são disciplina: nenhum hook do kit exige ID (o de mensagem cobra só o formato). Não sobe a 5: num clone novo nenhum hook existe, ele falha aberto sem Python, e “0 pulos declarados” não é “0 pulos”.
C4	papéis	3	Rebaixado de 4 para 3. A máquina valida o <i>esquema do documento</i> da skill, não o papel em execução — e o escopo_hook.py, único que restringiria de verdade, não está ligado no settings.json deste repositório. Verifiquei: só o hook do portão está. Some-se que 11 das 24 skills nunca dispararam.
C5	prova de eficácia	2	Existe um medidor automático que lê o git, e ele é honesto: declara que não mede se o kit ajudou . Medir conformidade não é medir eficácia. Um projeto, auto-medido, sem grupo de controle, é o degrau mais baixo da escala de evidência.
C6	custo de adoção	1	O número mais duro do documento: 62,6% dos commits do projeto real tocam só processo (117 de 187), contra 4 que tocam só produto. São 5 registros, 24 skills e um ritual de fechamento por sessão. O portão em si é barato: de 2 a 5 s por commit no projeto real.
C7	portabilidade	2	O conteúdo é portátil (markdown, Python, git). O <i>mecanismo</i> não é: CLAUDE.md, as 24 skills em formato SKILL.md e os hooks PreToolUse são do Claude Code. Trocar de ferramenta preserva os registros e perde as travas — a parte que faz o kit valer.
C8	honestidade sobre limites	3	Rebaixado de 4 para 3. Há cobrança real e ela surpreendeu o auditor (um teste reprova se o README divergir da contagem do script; outro reprova se o relatório perder a seção de limites) — mas o que a máquina cobra é a presença de uma <i>string</i> . Nenhum teste verifica se a lista de limites é completa ou verdadeira, e quem a escreve é quem se avalia.
C9	manutenção e comunidade	1	Uma pessoa. 43 commits em 11 dias, nenhuma tag, nenhum release, nenhum usuário além do dono. Se essa pessoa parar, o kit para.
C10	segurança e isolamento	2	Varredura de segredo no portão e um hook de escopo que nem está instalado e falha aberto por desenho. Sem sandbox de sistema operacional, sem allowlist de permissão própria. O isolamento que existe é o do Claude Code embaixo, não o do kit.
C11	equipe	2	Corrigido de 1 para 2. Existe caminho de atualização — new_project.py --upgrade, com marca de versão desde a v9 e manifesto que protege o que o dono customizou desde a v10. Ele rodou uma vez no projeto real, em 28/08 (v13.1 → v13.12), e parou ali: o projeto ainda roda o modelo de achado que o QA-15 da v13.16 consertou. Depende de alguém lembrar — e hook de git não viaja com o clone.
C12	desfazer e recuperar	2	Só o git, com a disciplina de ID no assunto tornando o histórico navegável. Sem checkpoint por passo, sem desfazer de um turno do agente.

Onde o kit perde feio

Sete dos doze critérios foram escritos esperando que o kit perdesse. Ele perdeu em todos os sete. Esta seção existe porque um benchmark em que o autor ganha em tudo não mediu nada.

1. Fator ônibus 1

Uma pessoa escreveu, mantém e usa o kit. Nenhuma tag, nenhum release, nenhum usuário externo, nenhum segundo par de olhos. Contra isso, Codex CLI, Copilot, Cursor, Claude Code e Gemini CLI têm equipe, cadência de release semanal e canal de suporte. **Se a pessoa sai de férias, o processo não tem quem o corrija; se muda de emprego, ele morre com o laptop.**

2. Distribuição em equipe: o caminho existe, foi percorrido uma vez e parou

O kit tem como atualizar um projeto — `--upgrade`, com marca de versão e, desde a v10, manifesto que protege o que o dono customizou. Ele rodou uma vez: em 28/08 o projeto real subiu da v13.1 para a v13.12. E parou ali — o projeto ainda carrega o defeito que fazia todo achado nascer contado como fechado, com o conserto pronto no kit desde a v13.16. E hook de git não viaja com o clone: no projeto real roda **uma trava de quatro**, e nada avisava. O Cursor guarda as regras no servidor e o administrador pode marcá-las como obrigatórias, sem o membro conseguir desligar; o Claude Code instala plugins de escopo administrado que não podem ser modificados; o Copilot impõe política pela organização no GitHub. Aqui, cada clone liga as próprias travas, ou fica sem elas.

3. Segurança e isolamento: o kit não tem, e um dos hooks nem está ligado

Codex CLI, Cursor, Claude Code e Gemini CLI têm sandbox de sistema operacional de verdade — Seatbelt no macOS, Landlock e seccomp no Linux. O kit tem varredura de segredo no portão e um hook de escopo que **falha aberto por desenho e não está instalado nem aqui nem no projeto real** — verifiquei o `settings.json` depois que o auditor apontou. O que protege a máquina é a ferramenta embaixo, não o kit.

4. Custo: 62,6% dos commits não entregaram produto

No projeto real, **117 de 187 commits tocam apenas processo** e **4 tocam apenas produto** — razão de 29 para 1. Some-se que 11 das 24 skills nunca dispararam. Nenhuma das 13 alternativas sobreviveria a esse número publicado sobre si, e o documento só pode publicá-lo porque foi o próprio kit que o mediu.

5. O ADR resolve a nossa maior tese por quase nada

A “lista-morta” não é nossa invenção, e a alternativa custa perto de zero.

O ecossistema ADR — o formato MADR em particular — tem campo formal de *Considered Options*, prós e contras por opção e *Decision Drivers*; a supersessão preserva o registro antigo em vez de apagá-lo. O cético encarregado de avaliá-lo sem condescendência escreveu que **na representação da rejeição o ADR empata ou ganha de quase todo mundo**, por custo próximo de zero.

Ele levou C2=3 e o kit levou C2=3 — **empate**, depois que o auditor rebaixou o nosso. O motivo do rebaixamento é exato: a nossa máquina cobra a integridade do *ponteiro* (ID que não existe, ID duplicado), não o *ato* de registrar. Nos dois casos, escrever a rejeição é disciplina humana.

A evidência independente — e ela aponta contra nós

A regra da casa manda publicar o resultado desfavorável. Um crítico foi encarregado de procurar medição independente sobre se processo pesado ajuda ou atrapalha no desenvolvimento assistido por IA, com ordem explícita de publicar o achado **mesmo sendo contra o kit**. Ele achou, e é contra.

Não existe hoje nenhuma evidência controlada e publicada de que uma camada de processo pesada e auto-autorada melhore o desfecho.

Os dois estudos que medem **diretamente** o mecanismo central do kit encontram efeito nulo sobre sucesso, com custo mensurável. Isto está na coluna do kit, e não só na dos concorrentes — a rodada anterior citava um destes estudos contra o Cursor e contra o AGENTS.md, e não contra si mesma. Usar um estudo como arma e não como espelho é seletividade de evidência, e um gestor descobre isso numa busca de trinta segundos.

Contra o kit

- **Gloaguen et al., ETH Zurich — Evaluating AGENTS.md (arXiv:2602.11988)**. Arquivos de contexto de repositório não melhoram, em geral, a taxa de sucesso, e aumentam o custo de inferência em mais de 20% em média. Mas a frase decisiva é outra: **instruções são bem seguidas; panoramas do repositório não são úteis**. O nosso **CLAUDE.md** é instrução — sustentado. O nosso **a_context_source.md**, que guarda estado, versão e métricas, é panorama — **não sustentado**, e é justamente o arquivo que o orçamento de 4.000 caracteres protege.
- **McMillan (arXiv:2605.10039)**, 1.650 sessões de Claude Code: tamanho, posição e conflito no arquivo de configuração **não tiveram efeito estatisticamente significativo** sobre adesão. O que teve efeito foi o comprimento da sessão — variável que o kit não gerencia. É preprint de autor único, não revisado por pares; é também o único estudo que testa a variável exata do nosso único mecanismo falseável.
- **METR (arXiv:2507.09089)**: 16 desenvolvedores experientes previram 24% de aceleração, relataram 20% depois de trabalhar, e ficaram **19% mais lentos**. **Ressalva obrigatória**: a atualização de fevereiro de 2026 revisou para -18% e -4%, com intervalos que cruzam o zero, e a própria METR passou a dizer que hoje é provável que haja ganho. Citar “19% mais lentos” como fato de 2026 seria a mesma seletividade que este documento condena. **O que sobrevive** é devastador para nós: a medição mais rigorosa que existe diz que o julgamento do praticante sobre a própria produtividade erra por larga margem, no sentido otimista — e toda a evidência de eficácia do kit é o julgamento do autor sobre o próprio kit.
- **DORA — AI Capabilities Model**. Das sete capacidades que o maior corpo de evidência sobre equipes com IA identifica, **uma** corresponde ao que o kit faz: práticas fortes de controle de versão. Registro de decisões, skills de papel, orçamento de contexto e padrão de mensagem de commit **não estão na lista**.
- **Buchgeher et al., IEEE Access 2023** (900+ repositórios): metade dos projetos que adotam ADR param entre 1 e 5 registros; média 6,03. Os 112 registros do kit em 11 dias são cerca de 18 vezes a média — com portão de máquina e o autor mais motivado que o kit jamais terá. Os dois fatos importam: o portão parece funcionar, **e** a amostra é a mais favorável possível.

A favor do kit

- **He et al., CMU (arXiv:2511.04427)**: projetos que adotaram Cursor tiveram ganho de velocidade grande e **transitório** e aumento **persistente** de avisos de análise estática e complexidade. É exatamente o modo de falha que um portão pre-commit ataca.
- **DORA 2025/2026**: IA correlaciona com mais throughput e mais instabilidade; o imposto de instabilidade é calculado em centenas de milhares de dólares quando a taxa de falha de mudança sobe um ponto. **GitClear (211 milhões de linhas)**: duplicação de blocos oito vezes maior, refatoração 39,9% menor.
- **Chroma — Context Rot**: o desempenho degrada com o comprimento da entrada bem antes de a janela encher. Sustenta manter o contexto pequeno como **controle de custo e de degradação** — não como controle de aderência.

O saldo, dito sem manobra.

Do que o kit implementa, **o portão de máquina é a única peça com apoio empírico independente** — e é a peça que o próprio repositório do kit contorna em 25 dos últimos 30 commits. As 24 skills de papel, os 5 registros e o orçamento de contexto **como ritual de governança** não têm medição favorável em lugar nenhum.

O que os revisores disseram, e o que eu apliquei

Duas revisões adversariais correram sobre as notas: uma encarregada de derrubá-las, outra de achar o que faltava. A regra de aplicação foi combinada antes: **correção de lógica se aplica; correção de fato de quem não pesquisou fica registrada como contestada**.

Aplicado

- **Riscar a posição no ranking.** Com 5 das 13 não auditadas, dizer “o kit é o N° X” seria afirmar uma precisão que o dado não tem. O documento não diz mais em que lugar o kit ficou.
- **Separar C1–C8 de C9–C12.** Quatro critérios mediam, no fundo, a mesma coisa — se existe uma empresa por trás — e contra isso um projeto de uma pessoa perde quatro vezes pelo mesmo motivo. A tabela dividida virou a página mais informativa do documento.
- **Marcar as não auditadas com asterisco** e publicar a assimetria da auditoria em número.
- **Publicar o empate impossível** entre Cursor e Claude Code, em vez de desempatar por conta.
- **Aceitar os três rebaixamentos do nosso kit** (C2, C4 e C8), inclusive o mais incômodo: o hook de escopo, que eu contava como mecanismo, **não está instalado**. Fui verificar o settings.json e o auditor estava certo.

Contestado — registrado, não aplicado

- O revisor hostil **não recebeu o texto dos doze critérios**, só os vetores de notas, e deduziu os eixos pelo padrão. Isso é **defeito do meu prompt**, não dele. Consequência: ele trocou C1 com C3 na argumentação. Os argumentos são bons e foram remapeados para o critério certo; o que dependia do rótulo errado não foi aplicado.
- Ele propôs **aplicar um deságio de -1 ponto** a cada ferramenta não auditada. Recusado: seria inventar um número para simular precisão. Preferi declarar a lacuna e marcá-la.
- Ele afirma que AGENTS.md e o ADR mereciam 3 ou mais em “desfazer e recuperar”, por herdarem o git. É um bom argumento de calibração, mas é **[suposto]** — ele não pesquisou — e mexeria em notas que nenhum cético dedicado revisou. Fica como item da rodada 3.
- A crítica de completude aponta que o número “138 issues, 12 repositórios”, atribuído ao estudo do ETH, **não aparece** no resumo do artigo nem na página do laboratório. Rebaixei a menção a **[suposto]** em vez de repeti-la como fato.

As treze alternativas

Uma ficha por alternativa, da maior para a menor pontuação em capacidade de processo (C1–C8). O texto de “o que é” foi escrito para quem nunca ouviu falar da ferramenta.

Aider — 28/40 em processo, 40/60 no total · *linha de base: o kit roda em cima dela*

Aider é um programa de linha de comando, gratuito e de código aberto, que conversa com um modelo de IA e edita os arquivos do seu projeto direto no disco. Ele lê o repositório com um analisador de código (tree-sitter) e monta um "mapa" resumido das classes e funções, para o modelo saber o que existe no projeto sem precisar ler tudo. A cada alteração aceita, ele faz sozinho um commit no git com mensagem escrita pela própria IA, e o comando `/undo` desfaz o último commit dele — o git vira o botão de "voltar atrás". Regras de estilo do time ficam num arquivo de texto (CONVENTIONS.md) que você manda o Aider ler como material só-de-leitura.

Forte em: Desfazer de verdade: commit automático por alteração, com mensagem gerada e atribuição no git, mais commit prévio do que já estava sujo — a IA nunca mistura o trabalho dela com o seu, e `/undo` reverte o último passo dela sem git manual. · Contexto derivado do código, não escrito à mão: o repo map por tree-sitter com ranqueamento em grafo e orçamento de tokens em código nunca fica desatualizado nem vira peso morto, porque é regerado a partir do próprio repositório. · Portabilidade total e prova pública acima da média do mercado: Apache 2.0, qualquer LLM (inclusive local), qualquer IDE, mais um arnês de benchmark aberto e resultados de desfecho publicados com custo e taxa de erro.

Fraco em: Nenhum portão que bloqueie: o lint roda sozinho mas só oferece corrigir, e o commit automático usa `--no-verify` por padrão, ou seja, atropela os hooks de pre-commit que a equipe já tem no repositório. · CONVENTIONS.md não é carregado sozinho nem cobrado: precisa de `/read`, `--read` ou de uma linha no `.aider.conf.yml`, e nada verifica se a regra foi seguida — é texto no prompt, e a própria referência de opções não tem a flag `--conventions-file` que terceiros citam. · Zero isolamento (sem sandbox, sem allowlist, `shell=True`, instala pacote em tempo de execução) e manutenção desacelerando em cima de uma pessoa só, com 1.838 issues abertas.

OpenAI Codex CLI — 27/40 em processo, 41/60 no total · linha de base: o kit roda em cima dela

É o programa de linha de comando da OpenAI que coloca um assistente de IA dentro do terminal para ler, escrever e rodar código no seu próprio computador. Ele lê um arquivo de instruções do projeto chamado AGENTS.md, pede sua aprovação antes de agir (ou não, se você mandar), e roda os comandos dentro de uma "caixa de areia" do sistema operacional que limita o que ele pode tocar fora da pasta do projeto. O código é aberto (Apache-2.0) e funciona no macOS, Linux e Windows, mas a proteção é mais forte no macOS e no Linux.

Forte em: Isolamento de sistema operacional real e inspecionável: Seatbelt no macOS e bubblewrap+Landlock+seccomp no Linux, deny-by-default, rede bloqueada por padrão em workspace-write, .git/ e .codex/ automaticamente somente-leitura para impedir escalada via git hooks, e comandos `codex debug seatbelt` / `codex debug landlock` que mostram o que está sendo aplicado. · Portão duplo cobrado por máquina: hooks PreToolUse/PermissionRequest podem negar a execução de Bash, apply_patch e ferramentas MCP, e requirements.toml deixa o admin travar política de aprovação, modo de sandbox e allowlist de MCP sem que o desenvolvedor consiga contornar. · Código aberto (Apache-2.0), provedores de modelo plugáveis (ollama, LM Studio, endpoint próprio) e artefatos de processo em padrões abertos (AGENTS.md, SKILL.md) que outras ferramentas leem, sobre uma base com manutenção e adoção enormes.

Fraco em: Desfazer é o buraco: o /undo existiu e foi retirado, Esc-Esc rebobina a conversa mas deixa os arquivos alterados, e há relatos de arquivos não versionados sobrescritos sem recurso — a recuperação é git manual. · No Windows o isolamento é o mais fraco e o mais quebradiço: exige setup próprio que falha com frequência (vários issues abertos), política corporativa costuma bloquear a instalação, e quando o modo elevated falha ele cai sozinho para unelevated, com isolamento de rede pior. · Contexto e memória são frágeis: o AGENTS.md é truncado em 32 KiB sem aviso no TUI, a compactação automática perde regras do AGENTS.md no meio da tarefa, e as memórias são por usuário, sem sincronizar entre máquinas e indisponíveis no EEE, Reino Unido e Suíça.

GitHub Copilot custom instructions — 23/40 em processo, 39/60 no total

É o jeito de "ensinar as regras da casa" ao GitHub Copilot escrevendo arquivos de texto simples dentro do próprio projeto: um arquivo geral (.github/copilot-instructions.md), arquivos que só valem para certas pastas (*.instructions.md com um filtro `applyTo`), receitas de tarefa reutilizáveis (*.prompt.md) e "personagens" com ferramentas limitadas (*.chatmode.md / *.agent.md). Como os arquivos ficam no repositório, todo mundo da equipe recebe as mesmas regras ao clonar o projeto, e o dono da organização ainda pode definir instruções e políticas que valem para todos os repositórios de uma vez. Importante: são instruções que influenciam o assistente, não travas — nada impede o Copilot de gerar código que as contradiz.

Forte em: Distribuição em equipe sem passo de instalação e com política por cima: os arquivos chegam a todos pelo git, instruções de organização (GA abr/2026) valem para todos os repositórios, políticas de empresa cascadeiam, e Agent Plugins 1.0 dá pacote versionado num marketplace. · Isolamento de execução real e configurável por organização: firewall de negação por padrão com allowlist no agente de nuvem, aprovação por ação e restrição de ferramentas por agente na IDE. · Documentação que declara limites por escrito, incluindo negativas incômodas (não vale para autocompletar inline, não determinismo, matriz de suporte por superfície, buraco do firewall em MCP e setup steps).

Fraco em: Nenhum portão: as instruções são viés e não política — o Copilot não valida a própria saída contra elas, e o code review dele comenta mas não bloqueia merge nem pede mudanças. · Comportamento inconsistente entre superfícies do próprio produto: instruções de organização não alcançam as IDEs, CLI e VS Code discordam sobre AGENTS.md, e há relato aberto e sem resposta de code review que ignora as instruções por completo. · Sem orçamento de contexto: nada limita o tamanho dos arquivos, que são anexados a cada requisição — custo de inferência medido em +20% e nenhum mecanismo de poda ou arquivamento quando o projeto cresce.

Gemini CLI — 23/40 em processo, 38/60 no total · linha de base: o kit roda em cima dela · notas 4+ não auditadas

É um programa do Google que você roda no terminal e que conversa com os modelos Gemini para ler, escrever e executar coisas no seu projeto — como um programador assistente que enxerga suas pastas e roda comandos. Você deixa instruções permanentes num arquivo de texto chamado GEMINI.md (um geral seu, um do projeto, um de cada subpasta), cria atalhos próprios em arquivos TOML, e pode prender o agente dentro de um contêiner Docker ou de um sandbox do macOS para ele não estragar nada fora dali. O código é aberto, mas atenção: desde 18 de junho de 2026 o Google desligou o atendimento para contas individuais (grátis, AI Pro e AI Ultra) e empurrou essas pessoas para um sucessor fechado chamado Antigravity CLI; hoje o Gemini CLI só continua funcionando com chave de API paga ou licença corporativa.

Forte em: Isolamento de execução de verdade e em camadas: contêiner (Docker/Podman), gVisor com kernel em espaço de usuário, LXC/LXD e Seatbelt nativo do macOS com seis perfis — somados a allowlist de ferramentas, Trusted Folders e concessão de ferramentas por subagente. · Desfazer com precisão de passo: repositório git sombra fora do git do projeto, snapshot antes de cada ferramenta que escreve, e /restore que recupera arquivos e conversa juntos e ainda re-propõe a chamada. · Aberto sob Apache 2.0 com cadência publicada (estável semanal, noturno diário), 106,8k estrelas e commits ativos — quem quiser pode bifurcar e continuar sem pedir licença ao Google.

Fraco em: Aprisionamento consumado: só fala com modelos Gemini e, em 18/06/2026, o Google desligou contas individuais (grátis, AI Pro, AI Ultra) e migrou todo mundo para um sucessor FECHADO em Go, deixando o repositório aberto em modo de manutenção para clientes corporativos. · Memória de decisão é só um markdown livre — nenhum esquema, campo ou validação para decisão e alternativas rejeitadas; nada distingue GEMINI.md bem cuidado de GEMINI.md abandonado. · Zero prova de desfecho: os benchmarks divulgados medem os modelos Gemini, não o processo da ferramenta, e não há score publicado do Gemini CLI como produto.

Cursor Rules e Memories — 22/40 em processo, 38/60 no total

É o sistema de "instruções permanentes" do editor Cursor. Você guarda arquivos de texto numa pasta ``.cursor/rules`` (extensão ``.mdc``) dizendo como o agente de IA deve trabalhar naquele projeto, e um cabeçalho em cada arquivo decide quando aquela regra entra: sempre, só quando você mexe em certos arquivos, ou quando o próprio modelo julga que precisa. Junto vem o recurso de "Memories", em que o Cursor anota sozinho fatos que apareceram nas conversas (você aprova antes de salvar) para lembrar deles depois — mas essas anotações ficam só na sua máquina/conta, não no repositório, e a equipe não as vê. Como está embutido no editor, ele traz de brinde coisas que um arquivo de regras sozinho não faz: checkpoints para desfazer o que o agente mexeu, hooks que podem barrar comandos, subagentes com ferramentas limitadas e um sandbox de sistema operacional.

Forte em: A plataforma dá de graça o que um kit de processo em arquivos não consegue dar: sandbox de SO real, hooks que negam chamada de ferramenta com exit code 2, subagentes com lista de ferramentas proibidas e checkpoints automáticos. · Distribuição em equipe resolvida no nível certo: Team Rules ficam no servidor, sincronizam sozinhas e o admin pode torná-las obrigatórias, sem depender de cada pessoa clonar a versão certa do processo. · Custo de entrada quase zero e seleção de contexto por glob, em vez de despejar tudo em todo prompt.

Fraco em: As regras não são cobradas por nada: entram no prompt e competem com o resto do contexto — o fórum está cheio de casos de ``alwaysApply`` ignorado, inclusive uma regressão confirmada na 3.1.x, e de glob-attach instável no modo agente. · Memories são o oposto de memória de decisão: pessoais, não versionadas, fora do repositório, sem diff, sem blame e sem rollback — quem entra na equipe começa do zero e cada pessoa acumula um conjunto diferente. · Prisão de fornecedor: formato ``.mdc``, memórias, checkpoints, hooks e subagentes só existem dentro do Cursor, e a própria empresa já quebrou o formato antes ao aposentar o ``.cursorrules``.

Claude Code nativo — 22/40 em processo, 38/60 no total · linha de base: o kit roda em cima dela

É o conjunto de recursos que já vem dentro do próprio Claude Code, sem instalar framework nenhum: um arquivo CLAUDE.md com as regras do projeto que o agente lê no começo de toda sessão; "skills", que são arquivos de instrução carregados só quando fazem falta; "hooks", que são scripts seus que o programa executa sozinho em momentos fixos (antes de uma ferramenta rodar, depois de um arquivo ser editado) e que podem barrar a ação; "subagentes", que são trabalhadores separados com ferramentas limitadas e contexto próprio; e "plugins", que empacotam tudo isso para instalar e versionar em várias pessoas. Junto vem um sistema de permissões (listas de permitir/perguntar/negar por ferramenta e por caminho) e um sandbox de sistema operacional que limita quais arquivos e quais domínios de rede os comandos de shell podem tocar. A documentação é incomum por dizer, com todas as letras, onde cada peça falha: o CLAUDE.md é "contexto, não configuração aplicada", e o sandbox "não é uma fronteira de isolamento completa".

Forte em: Honestidade documentada acima da média do mercado: a doc afirma por escrito que CLAUDE.md é contexto e não configuração aplicada, que o sandbox não é fronteira completa de isolamento, e que hook em timeout não serve de portão — e ainda aponta o mecanismo certo (hook PreToolUse ou regra deny) para quem quer garantia de verdade. · É a única camada de processo avaliada que combina portão mecânico de chamada de ferramenta, isolamento de SO real (Seatbelt/bubblewrap+seccomp com allowlist de rede), limite de ferramentas por subagente e política de organização que o desenvolvedor não consegue sobrescrever. · Custo de adoção quase zero e escada de escala embutida: já vem na ferramenta, ``/init`` gera o arquivo de regras, e há progressão natural (regra → skill → hook → plugin) com medição de custo de contexto por skill e por plugin.

Fraco em: A camada que carrega o processo (CLAUDE.md, skills) é a menos garantida de todas: entra como mensagem de usuário, sem compliance garantido, e a única saída documentada quando a regra precisa valer é reescrevê-la como hook — ou seja, boa parte do 'processo' continua sendo persuasão. · Não há registro de decisão digno do nome: nada captura alternativas rejeitadas, a auto-memória é local da máquina e não viaja para a equipe, e não encontrei esquema nem validação para esse tipo de conteúdo. · Trava total de fornecedor na camada de mecanismo: hooks, subagentes, plugins, marketplaces, permissões e sandbox não existem fora do Claude Code, e a ponte com AGENTS.md é manual e só de entrada.

AWS Kiro — 22/40 em processo, 38/60 no total

Kiro é um programa da Amazon (AWS) para escrever código com ajuda de inteligência artificial. A diferença dele para os concorrentes é que, antes de escrever qualquer linha de código, ele obriga a produzir três documentos: o que o sistema precisa fazer (requirements.md), como será construído (design.md) e a lista de tarefas na ordem certa (tasks.md) — só depois os agentes executam. Vem em três formatos (um editor próprio parecido com o VS Code, uma versão de linha de comando e uma versão no navegador), é pago por créditos de uso, e traz "hooks": automações que disparam sozinhas quando um arquivo é salvo ou antes de o agente usar uma ferramenta.

Forte em: Portão de verdade cobrado por máquina: hooks com gatilhos bloqueantes (PreToolUse, PromptSubmit, PreTaskExecution) que param o agente no código de saída, somados a permissões por capacidade com efeito deny que nega a chamada de ferramenta e a agentes com lista de ferramentas restrita. · Infraestrutura de operação madura que kits artesanais raramente têm: checkpoint automático por turno com rebobinagem de conversa, compactação automática com medidor de contexto, governança corporativa com SSO/SCIM e allowlist central de modelos e servidores MCP. · O processo mora em arquivos de texto dentro de .kiro/ no próprio repositório — specs, steering, hooks e agentes chegam iguais a toda a equipe por git, sem instalador paralelo.

Fraco em: A especificação não guarda o que foi descartado: o design.md registra a escolha, não as alternativas rejeitadas nem o porquê, e há relato consistente de que requirements e design não se atualizam conforme o código anda (spec drift), transformando o artefato em documento morto. · Preso ao fornecedor e ao medidor: produto de código fechado, uso pago em créditos com excedente a US\$0,04, tier gratuito de 50 créditos praticamente simbólico, histórico de bug de medição e conta-surpresa, e proibição explícita de usar através de harnesses de terceiros. · Nenhuma prova pública de desfecho: os ganhos citados são falas de executivo e estudos de caso do próprio fornecedor, sem estudo controlado, benchmark ou telemetria divulgada sobre qualidade, defeito ou tempo.

Devin — 22/40 em processo, 38/60 no total · linha de base: o kit roda em cima dela · notas 4+ não auditadas

Devin é um "programador de aluguel" que roda na nuvem: você abre uma tarefa por Slack, IDE ou navegador e ele ganha uma máquina virtual Linux só dele, onde clona o repositório, escreve código, roda os testes e abre um pull request sozinho. Para não errar sempre do mesmo jeito, ele tem duas gavetas de instruções que a equipe escreve: o Knowledge (dicas e convenções que ele lembra quando o assunto aparece) e os Playbooks (roteiros passo a passo para tarefas que se repetem, com critério de sucesso). É um produto fechado e pago por consumo — você não instala num arquivo do seu projeto, você aluga o serviço da empresa Cognition.

Forte em: Isolamento de verdade: VM Linux dedicada por sessão, sandbox de SO na CLI e regras de permissão deny/ask/allow com precedência que bloqueiam a chamada antes de executar — mecanismo, não recomendação · Governança de equipe cobrada por máquina: manifesto de plugins com required/optional/forbidden e hierarquia enterprise > org > repo > usuário, mais tetos de ACU por organização, por membro e por sessão que param o agente ao estourar · Desfazer por passo: restauração de checkpoint reverte arquivos e memória do agente pela linha do tempo da sessão, coisa que a maioria das alternativas deixa para o git manual

Fraco em: Aprisionamento total ao fornecedor: Knowledge e Playbooks moram no painel da Cognition, sem exportação documentada, e o processo não sai de lá para outra IDE ou modelo · Custo imprevisível por sessão: ACU sobe com tarefa mal definida (o agente explora, volta atrás e refaz testes queimando crédito) e você paga igual quando o resultado é descartado · O portão de processo é fraco justamente onde importa: os lifecycle hooks são declarados 'best effort and fail open', e nada verifica mecanicamente se o Playbook foi cumprido

BMAD-METHOD — 21/40 em processo, 34/60 no total · notas 4+ não auditadas

É um conjunto de instruções em markdown que instala no seu projeto uma "equipe" de agentes de IA com papéis fixos — analista, gerente de produto, arquiteto, UX, scrum master, desenvolvedor e QA — cada um com seu roteiro de trabalho. Em vez de você pedir código direto para a IA, você passa por etapas: primeiro um documento de requisitos (PRD), depois um documento de arquitetura, que são fatiados em histórias pequenas, e só então o agente desenvolvedor implementa uma história por vez. A ideia é que o documento vire a fonte da verdade e o código seja consequência dele; o preço disso é um processo longo e caro em tokens.

Forte em: Papéis especializados de verdade e empacotados: doze agentes nomeados com workflows próprios, entregues como skills e plugins versionados, com override por `.customize.yaml` que sobrevive ao upgrade · Portabilidade e distribuição maduras: MIT, compila para mais de 30 IDEs de agente, com pacote npm e marketplace de plugin no Claude Code e no Codex — o time instala e atualiza por comando · O `bmad-loop` traz orquestração determinística em Python de verdade: gates de dependência que recusam despachar a story no preflight e no dispatch, orçamento de retry, worktree por story e reversão automática ao baseline quando a verificação falha

Fraco em: Peso por sessão alto e documentado pelos próprios usuários: 82,1k e 96,5k tokens num único passo de análise, ~US\$200 por ciclo completo, 12 a 16 horas de planejamento antes da primeira linha de código · Os portões do fluxo padrão são julgados pelo LLM, não cobrados por máquina — há registro de story entregue com stub vazio e comentário TODO passando como resolvida, e de review que força um mínimo artificial de três achados · A documentação principal não declara custo, limites nem quando não usar, e a própria solução de contexto (sharding) foi desaconselhada pela doc enquanto uma issue aberta afirma que ela nunca economizou contexto

GitHub Spec Kit — 21/40 em processo, 29/60 no total

É um kit oficial do GitHub que obriga o agente de IA a escrever documentos antes de escrever código. Você instala um comando (`specify`) no seu projeto e ele cria uma sequência de etapas em forma de comandos de barra: primeiro uma "constituição" com os princípios inegociáveis do projeto, depois a especificação do que se quer, depois o plano técnico, depois a lista de tarefas, e só então a implementação. Tudo isso vira arquivos markdown dentro do próprio repositório, que a equipe pode ler e revisar no Git. A ideia é trocar o "pedir e torcer" por um processo escrito em fases.

Forte em: Portabilidade real do resultado: os artefatos são markdown comum versionado no seu próprio repositório, sob MIT, com 30+ integrações de agentes — se você abandonar o Spec Kit amanhã, os documentos continuam úteis e legíveis. · Distribuição em equipe genuinamente empacotada: bundles com manifesto que fixa versão por componente, validação estrutural por comando, artefato `.zip` versionado e catálogos com política de instalação — bem acima da média de mercado, ainda que a atualização esteja com bug aberto. · Respaldo institucional e ritmo de manutenção incomuns: mantido pela própria GitHub, 133,6k estrelas, v1.0.0 estabilizada e releases quase diários até set/2026.

Fraco em: O portão não é portão: `/speckit.analyze` é um prompt que "recomenda" resolver antes de implementar, e a documentação de extensões afirma que hooks não podem bloquear nem falhar — a constituição é um LLM julgando outro LLM contra um markdown, e as três revisões independentes que li relatam justamente código que não obedece à spec. · Peso por sessão alto e sem orçamento cobrado: 18,6k tokens de comandos carregados sempre (~93% da janela padrão do Cursor), 2.577 linhas de markdown para ~989 linhas de código num teste real, e nenhum mecanismo de arquivamento ou poda quando o projeto cresce. · Isolamento negativo: além de não ter sandbox próprio, a integração grava auto-aprovação de terminal para os scripts do `.specify/`, afrouxando a proteção que o agente hospedeiro daria — e a issue que apontou isso, junto com um caminho de injeção de comando por nome de branch, foi fechada como "not planned".

Cline / Roo Code Memory Bank — 20/40 em processo, 35/60 no total · notas 4+ não auditadas

É um jeito de dar "memória" a um assistente de programação que esquece tudo entre uma conversa e outra. Você mantém uma pasta com alguns arquivos de texto comum (markdown) — o que é o projeto, por que ele existe, o que está sendo feito agora, o que já ficou pronto, quais decisões foram tomadas — e instrui o assistente a ler todos esses arquivos no começo de cada tarefa e atualizá-los no fim. Não é um programa que você instala: é um texto de instrução que você cola na configuração do Cline (ou do Roo Code) e alguns arquivos que o próprio assistente cria. À parte disso, o Cline grava um "ponto de restauração" do seu projeto a cada ação que executa, para você conseguir voltar atrás no código sem perder a conversa.

Forte em: Custo de entrada quase zero e portabilidade real: são arquivos markdown comuns mais um prompt, legíveis por humano e copiáveis para qualquer IDE ou modelo — o padrão vazou do Cline para praticamente todo o mercado. · Checkpoints do Cline são o melhor item do pacote: git sombra com snapshot a cada chamada de ferramenta, incluindo arquivos fora do git, com diff e restauração separada de arquivos e de conversa. · Os modos do Roo Code impõem escopo por máquina: a restrição fileRegex devolve FileRestrictionError e nega a edição, em vez de apenas pedir que o agente se comporte.

Fraco em: Zero cobrança automática: nada verifica se o agente leu ou atualizou os arquivos; o "MUST read ALL memory bank files" é prompt, e há issue registrada de o Cline afirmar ter atualizado os arquivos e só imprimir o conteúdo no chat. · Não escala: ler todos os arquivos em toda tarefa é orçamento crescente, e a documentação não oferece poda, arquivamento nem teto de tamanho. · Sem prova de eficácia e sem honestidade sobre limites: o material oficial não traz um único número de desfecho e não declara por escrito onde o padrão falha (memory bank desatualizado, custo de token, divergência entre pessoas).

Padrão aberto AGENTS.md — 17/40 em processo, 24/60 no total · notas 4+ não auditadas

É um acordo entre empresas de que todo projeto pode ter um arquivo de texto simples chamado AGENTS.md na raiz, escrito em Markdown, contando ao assistente de IA o básico do projeto: como instalar, como rodar os testes, que estilo de código usar. Funciona como um "leia-me para robôs" — em vez de cada ferramenta (Codex, Cursor, Copilot, Gemini CLI, Zed) exigir um arquivo com nome diferente, todas leem o mesmo. O ponto central, e mais incomum, é o que ele deliberadamente NÃO faz: não existe formato obrigatório, nem campos exigidos, nem validador. Você escreve o que quiser, com os títulos que quiser, e a ferramenta simplesmente joga aquele texto na conversa com o modelo. Não é um programa que você instala nem um processo que você segue — é só a combinação sobre o nome do arquivo.

Forte em: Portabilidade real e vendor-neutral: 20+ ferramentas leem o mesmo arquivo, sob governança da Linux Foundation (Agentic AI Foundation), cofundada por concorrentes diretos — é o item de menor risco de aprisionamento do mercado inteiro. · Custo de adoção quase nulo e adoção medida e publicada: um arquivo Markdown sem instalação, com 60 mil+ repositórios públicos e 24,2 mil estrelas, números que quase nenhum concorrente apresenta. · É a única alternativa avaliada com estudo controlado independente sobre desfecho (ETH Zurich, 138 repositórios, 4 agentes) — mesmo que o resultado seja desfavorável, existir a medição é uma força de transparência do ecossistema.

Fraco em: Zero cobrança por máquina em toda a linha: nenhum portão, nenhum esquema, nenhum validador, nenhum limite de tamanho no padrão. É 100% texto que o modelo pode ignorar — e há relatos consistentes de que ignora. · Nada de segurança ou isolamento, e pior: o próprio arquivo é vetor de ataque documentado (injeção de prompt com exfiltração de credenciais no Codex CLI), porque o conteúdo entra na conversa como instrução confiável, sem assinatura nem revisão. · Não cobre o trabalho: sem memória de decisão, sem papéis, sem desfazer por passo. Resolve apenas 'onde o agente lê o contexto', deixando organização, prova e recuperação inteiramente por conta de outra ferramenta.

Ecossistema ADR — 16/40 em processo, 23/60 no total

É o hábito de anotar cada decisão importante de arquitetura num arquivo de texto curto guardado junto com o código, criado por Michael Nygard em 2011. Cada arquivo diz o contexto, a decisão tomada e as consequências; quando a equipe muda de ideia, o arquivo antigo não é apagado, e sim marcado como "substituído". Em volta disso nasceram ferramentas: o adr-tools (scripts de terminal que numeram e marcam a substituição), o template MADR (que acrescenta campos para as opções consideradas e os prós e contras de cada uma) e o Log4brains (que transforma os arquivos num site pesquisável).

Forte em: Custo de entrada e portabilidade imbatíveis: um arquivo markdown no git, sem runtime, sem servidor, sem dependência de modelo ou IDE, licença MIT/CC0 — legível daqui a dez anos mesmo que todas as ferramentas morram. · O MADR 4.0.0 formaliza a lista de opções rejeitadas melhor que quase todo o mercado: campo 'Considered Options' mais 'Pros and Cons of the Options' com prós e contras por opção, e a supersessão preserva o registro antigo em vez de apagá-lo. Quem vende a 'lista-morta' como tese central precisa reconhecer que o ADR já entrega isso por praticamente nada. · Padrão de fato com 15 anos e vocabulário compartilhado na indústria — ThoughtWorks Radar no anel Adopt, 5.700 estrelas no adr-tools, 2.500 no MADR, organização própria em adr.github.io — e totalmente independente de qualquer fornecedor de IA.

Fraco em: Zero cobrança por máquina: não bloqueia commit, não falha CI por decisão ausente, não valida esquema, não nega nada. Tudo depende de disciplina humana — e a evidência empírica mostra o resultado: cerca de 50% dos repositórios com ADR param entre 1 e 5 registros, média de 6,03 por repositório, menos de 2% passando de 25. · Ferramentas de referência paradas ou dormentes: o adr-tools, o mais citado do ecossistema, está sem commit no master desde 30/03/2020 (quase seis anos, com 32 issues e 37 PRs abertos); o Log4brains está sem release no npm há mais de um ano. Só o MADR segue ativo — e é justamente o que não traz ferramenta própria. · Nenhum desenho para agente de IA: não controla o que é lido nem tem orçamento de contexto, não tem papéis com escopo executável, não isola execução, não reverte passo. E o template dominante no uso real é o do Nygard, que sequer tem campo para opções rejeitadas — a lista-morta formal está no MADR, que é minoria.

O que este documento não responde

Ele não responde se o kit ajuda.

Responder isso exigiria **o mesmo projeto construído sem ele**, pela mesma pessoa, no mesmo período. Esse projeto não existe. Enquanto não existir, todo número aqui descreve o que aconteceu **com** o kit — nunca o que teria acontecido sem — e a diferença entre as duas coisas é a pergunta inteira.

É a regra do próprio kit, aplicada a ele mesmo: *lacuna declarada fica declarada*.

Outras cinco coisas que ficaram de fora

- **Qualidade do que foi registrado.** Uma decisão pode estar completa, datada, dentro do teto — e errada. O portão conta registros; ele não os julga.
- **Se uma skill era aplicável.** “Nunca disparou” tem duas causas: não foi usada, ou não servia para este projeto. O script não sabe distinguir.
- **Quantos commits pularam o portão sem declarar.** O pulo declarado é contado; o pulo com o hook desligado segue invisível. Portão com taxa de contorno desconhecida não tem taxa de contorno igual a zero.
- **Quanto contexto uma sessão gastou de fato.** O kit mede o *tamanho do arquivo* que a regra manda ler. Quanto o agente realmente carregou é comportamento, não arquivo.
- **Como as 13 alternativas se comportam em uso.** Nenhuma foi rodada. Esta é a maior lacuna do documento e ela favorece quem foi só lido.

A recomendação

Para uma equipe de seis pessoas decidindo hoje, com base no que este documento conseguiu medir e no que a evidência independente diz. Ela não é “adotar o kit” nem “adotar uma das 13”.

Não adotar o kit como está

Fator ônibus 1, zero usuários externos, nenhum caminho de saída documentado, a única cópia parada na v13.12, 62,6% dos commits em processo — e as duas mecânicas que carregam a maior parte dos pontos (contexto e papéis) ficam exatamente onde a evidência publicada mede efeito próximo de zero. Multiplicar isso por seis multiplica o custo por seis e a garantia por zero.

Não adotar BMAD nem Spec Kit como fluxo principal

BMAD tem relato de 82,1 mil e 96,5 mil tokens num único passo, cerca de 200 dólares por ciclo e 12 a 16 horas de planejamento antes da primeira linha de código, com portões julgados por outro LLM. No Spec Kit, o comando de análise **recomenda** resolver antes de implementar, e a própria documentação de extensões diz que hooks não podem bloquear nem falhar. Portão que não bloqueia não é portão — é a cerimônia com o custo e sem a cobrança.

Adotar quatro peças, das quais só uma vem do kit

- **Base:** a ferramenta de agente que a organização já licencia. Escolha por licença existente e portabilidade, não por recurso: a camada de processo é fina e migra; a camada de fornecedor é grossa e não migra.
- **1. Portões no CI, não no prompt** — é a peça do kit que vale a pena, e a única com apoio empírico independente. **Leve a ideia do check.py e descarte a implementação:** porte as verificações para ferramenta padrão (framework pre-commit, linters, pytest), para que o laptop de quem entrar e o CI rodem a mesma coisa. Um portão de 78 KB com suíte própria de 141 testes que leva 13,7 minutos é um **segundo produto** para manter, com um mantenedor só.
- **2. Um arquivo de contexto por repositório**, mínimo, escrito por humano, com o nome nativo da ferramenta. Escreva **instruções e práticas não-padrão**; **não escreva panorama do repositório** — é a categoria que o estudo do ETH isola como não útil. Proíba arquivo de contexto gerado por IA: é a única coisa que a medição mostra sendo ativamente negativa. Mantenha um teto como **controle de custo**, implementado como aviso de lint, não portão.
- **3. ADR com piso, não com cerimônia.** MADR em **docs/adr**, um arquivo por decisão, com a seção de opções consideradas preenchida — é aí que a melhor ideia do kit (registrar as **rejeições**) transfere sem trazer o resto junto. Contra o decaimento medido por Buchgeher, acrescente **uma** checagem de CI: mudança que toca caminho arquitetural precisa citar um ADR.
- **4. Meça o desfecho antes de adotar qualquer coisa.** As quatro chaves do DORA — sobretudo taxa de falha de mudança e retrabalho — nas seis semanas **anteriores**. Sem linha de base a equipe termina na posição da METR: convencida de +20% enquanto está em -19%, sem jeito de saber.

Descartar, explicitamente

- **As 24 skills de papel.** Onze nunca dispararam em projeto nenhum, e proliferação de papéis não tem medição favorável em lugar algum.

- **Os 5 registros, reduzidos a 2.** Decisões (com as rejeições) e o rastreador de issues que a equipe já tem. 44 achados de QA e 15 questões em 11 dias são **issues**, não arquivos markdown paralelos que ninguém audita.
- **O hook que exige citar um ID na mensagem de commit.** O próprio repositório do kit o viola em 25 dos últimos 30 commits — é a medição de que uma regra que ninguém cumpre é uma regra que todo mundo contorna.

O piloto que este documento deveria ter proposto e não propôs

Seis semanas, duas das seis pessoas, um repositório.

Quatro chaves do DORA medidas seis semanas **antes** · critério de parada pré-registrado por escrito antes de começar · comparação contra as outras quatro pessoas em trabalho comparável.

E o custo de estar errado, dito em horas de salário no slide: no N=1 mais favorável possível, **62,6% dos commits foram processo**. Diga esse número vezes seis, em reais. É a única frase da apresentação que alguém vai lembrar na semana seguinte.